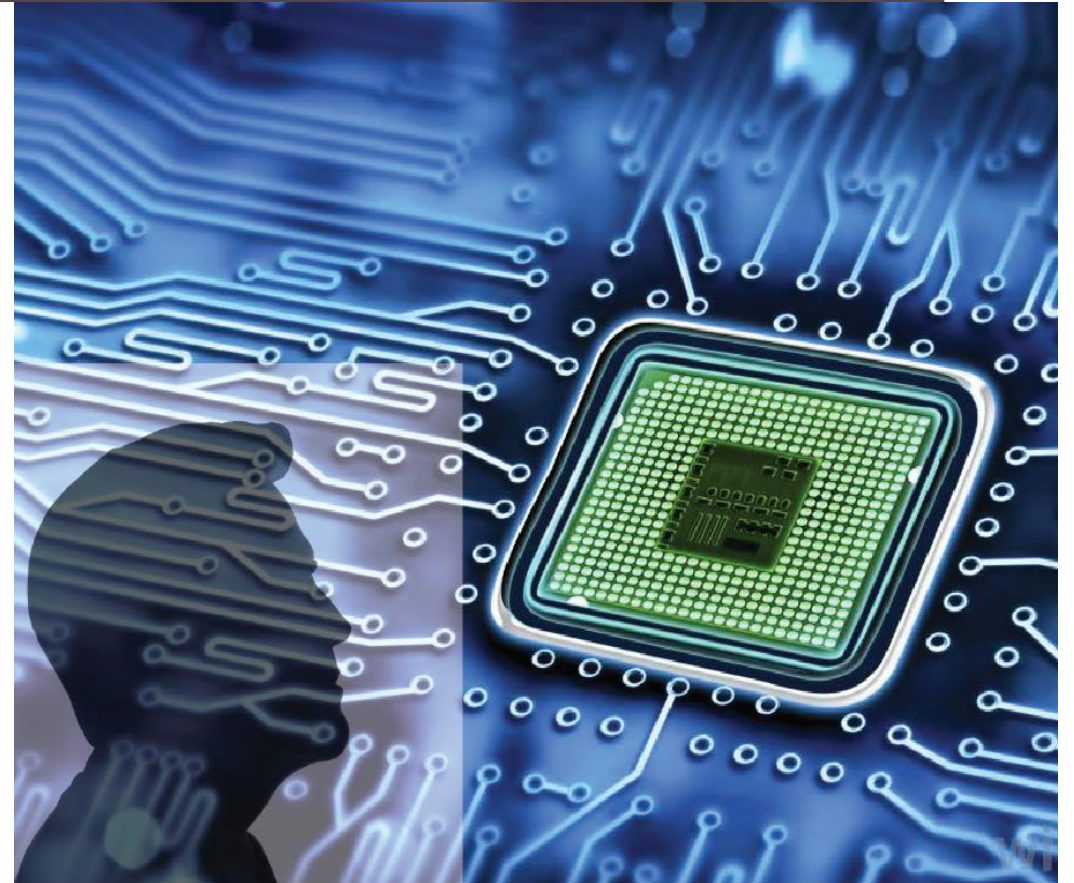


VLSI DESIGN FLOW

Week-11 Lecture-2 DFT

Sneh Saurabh
6th November, 2024



VDF: Revision/Announcements

-



Scan Design Flow: Number of cycles required

Problem:

Given:

Number of Test Vectors = T

Number of Scan Cells in Scan Chain = N

Calculate:

Number of Clock Cycles Required.

Solution:

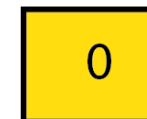
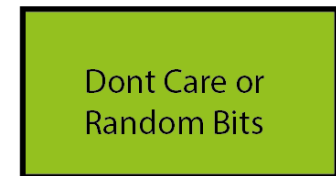
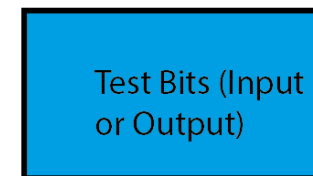
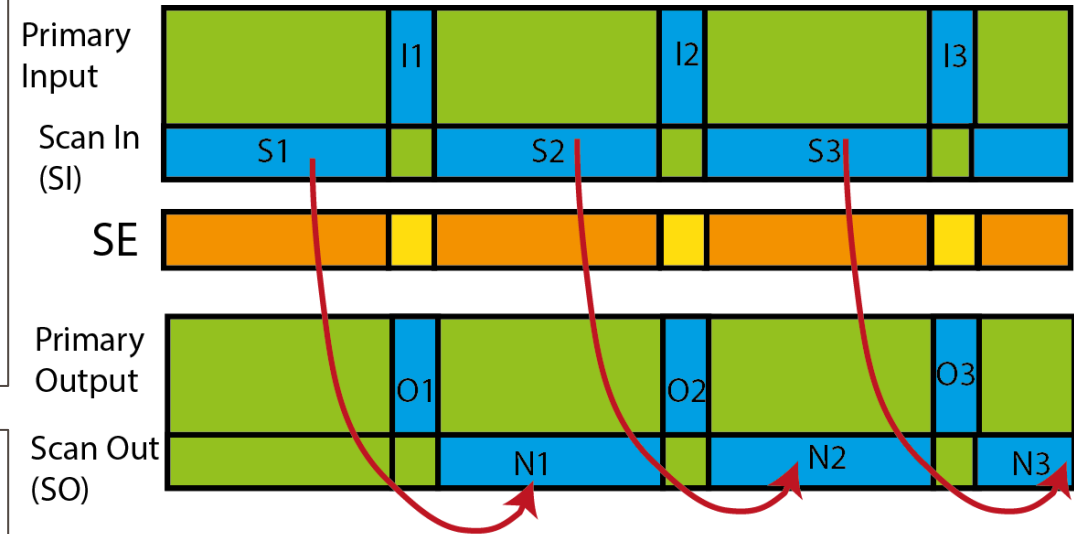
First Test vector: $N + 1 + N$

Second Test vector: $N + 1 + N$

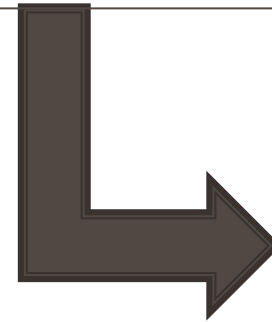
Third Test vector: $N + 1 + N$

.....

Total: $T(N + 1 + N) - (T - 1)N$
 $= TN + T + N$



Design for Test



ATPG

ATPG: Objective, Challenges and Practical Solution

ATPG: Automatic Test Pattern Generator

- **Objective:** Generate test patterns (test vectors) that can be used to detect faults
- **Desired:** small set of test vectors that detect all faults considered for that circuit

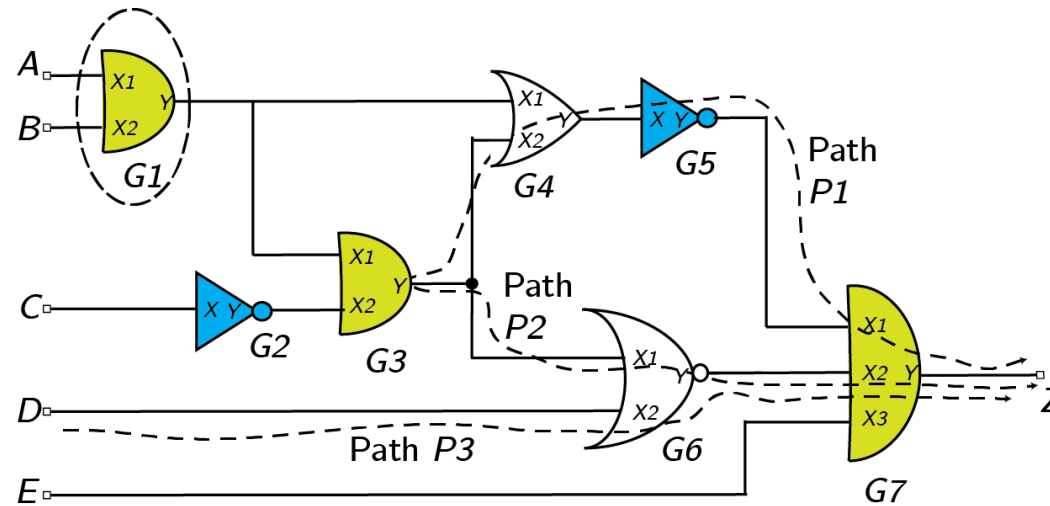
Challenges

- For generalized sequential circuits, generating test vectors/sequences is very difficult
 - Reaching certain states may take exponential clock cycles
- Finding the test sequence that controls or observes an internal point in the circuit is runtime intensive

Practical Solution

- Sequential ATPG problem is transformed to Combinational ATPG problem by Scan Design Flow
- Combinational ATPG problem, though NP-complete, has efficient algorithms

ATPG: Terminologies



- **Path:** sequence of pins in topological order
- **On-path input** or **on-input:** for a given path, the input pins lying on that path
- **Side path inputs** or **side-inputs:** The input pins other than the on-inputs of the instances lying on that path

ATPG: Controlling/Non-controlling values

Controlling value of a multi-input gate

- Value that can be assigned to any input of the gate such that the output is known irrespective of values on other inputs.

Non-controlling value of a multi-input gate

- Value that can be assigned to any input of the gate such that the output is decided by the value on other inputs

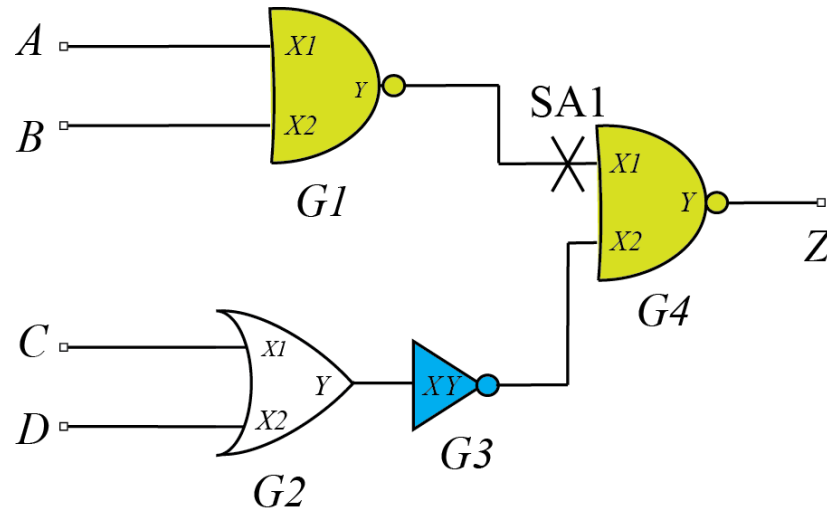
Type of Gate	Controlling Value
AND	0
NAND	0
OR	1
NOR	1
XOR	Not defined

ATPG: General Approach

Path Sensitization Method

- Fault Sensitization
- Fault Propagation
- Line Justification

ATPG: Illustration of Path Sensitization Method



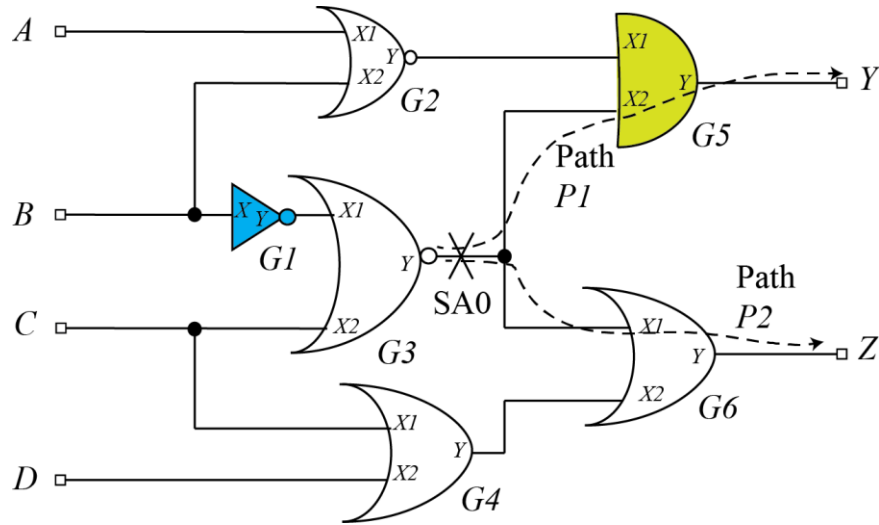
Given: Stuck-at 1 at $G4/X1$

To Find: A test vector (value of A , B , C , D) which will be able to detect the existence of $SA1$ at $G4/X1$

Test Vector: (1,1,0,0)

- **Fault Sensitization:** Find the value at input ports that will set the value at the fault site **opposite** of the fault value
 - Find A , B , C , D such that $G4/X1=0$ ($A=1$, $B=1$)
- **Fault Propagation:** For the path from the fault site to the output port, set the value of side-inputs to non-controlling inputs
 - Set $G4/X2$ to non-controlling value ($G4/X2=1$)
- **Line Justification:** Find the value at input ports that will set the side-inputs as found in fault propagation
 - Find A , B , C , D such that $G4/X2=1$ ($C=0$, $D=0$)

ATPG: Illustration of Backtracking



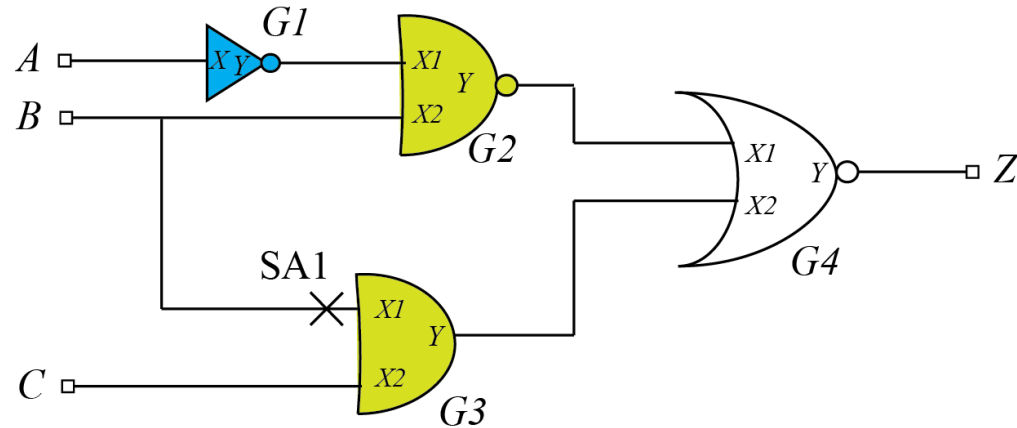
Given: Stuck-at 0 at $G3/Y$

To Find: A test vector to detect this fault.

Test Vector: (x,1,0,0)

- Fault Sensitization
 - Find A, B, C, D such that $G3/Y=1$ ($B=1, C=0$)
- Fault Propagation: **Path to Y**
 - Set $G5/X1$ to non-controlling value (i.e. 1)
- Line Justification: $A=0, B=0$
 - **B=0 is in conflict!**
 - Must be Backtracked
- Fault Propagation: **Path to Z**
 - Set $G6/X2$ to non-controlling value (i.e. 0)
- Line Justification: $C=0, D=0$

ATPG: Illustration of Redundant Fault



Given: Stuck-at 1 at pin $G3/X1$

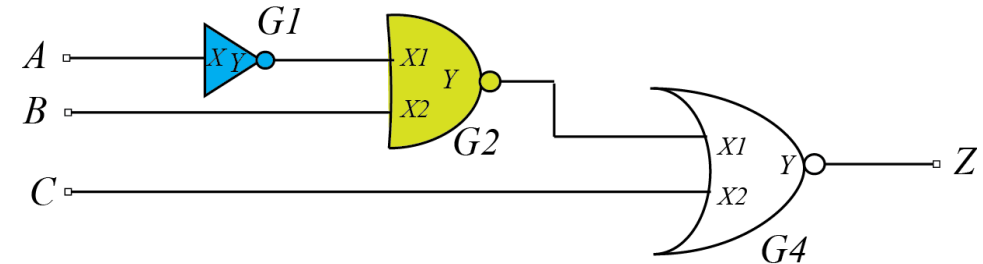
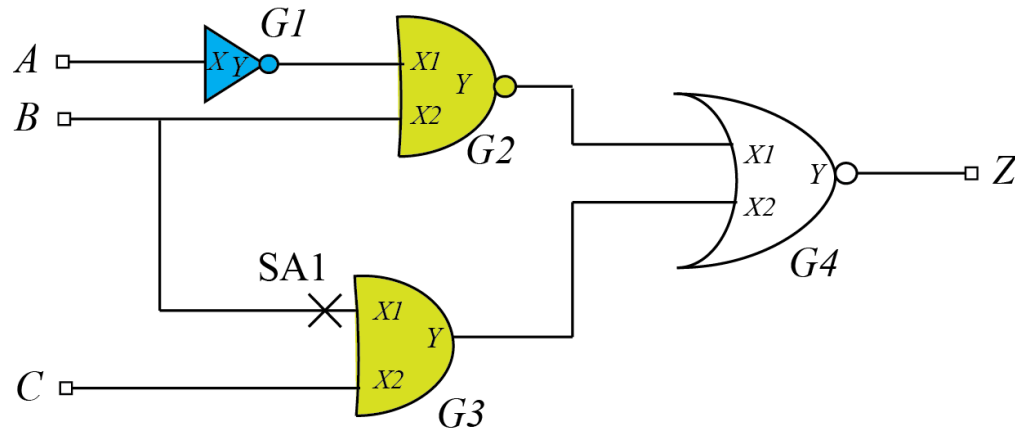
To Find: A test vector to detect this fault

- Fault Sensitization
 - Find A, B, C such that $G3/X1=0$ ($B=0$)
- Fault Propagation:
 - For Propagation to Z: Set $G3.X2=1$ and $G4/X1=0$
- Line Justification:
 - $B=1$ is in conflict with earlier $B=0$

- No test vector exists to detect this fault
- This is known as *redundant fault*

ATPG: Redundant Fault based Optimization

- Redundant faults can be used to reduce the hardware
- Redundant fault means that the behavior of the circuit with/without fault is the same
- Assume that the fault site has constant 0/1 optimize out gates that are not required



- Assume that SA1 at $G3/X1$ is always present.
- $G3/X1=1$ implies that $G3/Y=C$

ATPG: Challenges and Solutions

Complications

- Primarily due to re-convergent fanouts
- Decisions required to be taken at a particular point
- Decision leads to implications
- Some decisions can be wrong (leading to conflicts in decisions)
 - Backtracking required

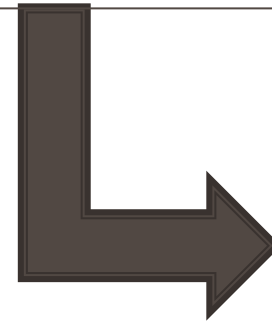
Backtracking Limit

- Too many backtrack => possible redundant fault
- User defined Backtrack Limit employed by tool
 - Example Backtrack Limit = 1000
 - Abort finding test vector when backtrack limit is hit, i.e. when 1000 times backtracking has been done

Algorithmic Advancements

- D-algorithm: Roth 1966
- PODEM (Path-oriented Decision Making): Prabhu Goel 1981
- FAN (Fanout-oriented) Algorithm: Fujiwara

Design for Test



BIST

Built-in Self Test (BIST)

Drawbacks of ATE-based testing:

- Cost of testing high
 - ATE cost
 - Voluminous test data: increase test time
- Can be done only in production test environment
 - Lacks capability of field testing
- At-speed test difficult
 - Probe's inductance/capacitance bottleneck

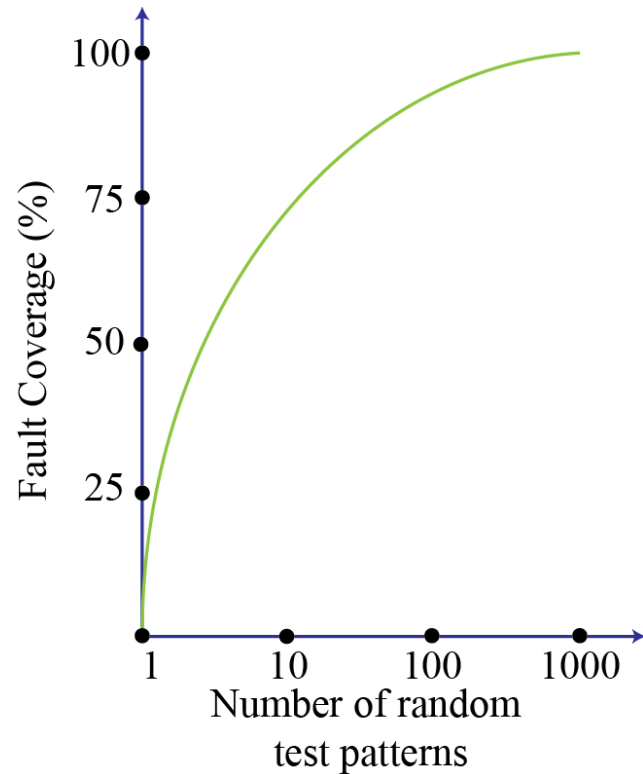
BIST as a solution:

- Test-oriented additional hardware and software features **inside a given IC**
- Allows IC to test their own operation
- Reducing dependence on an external ATE for testing
 - Repair in the field possible

Built-in Self Test (BIST): Distinguishing Features

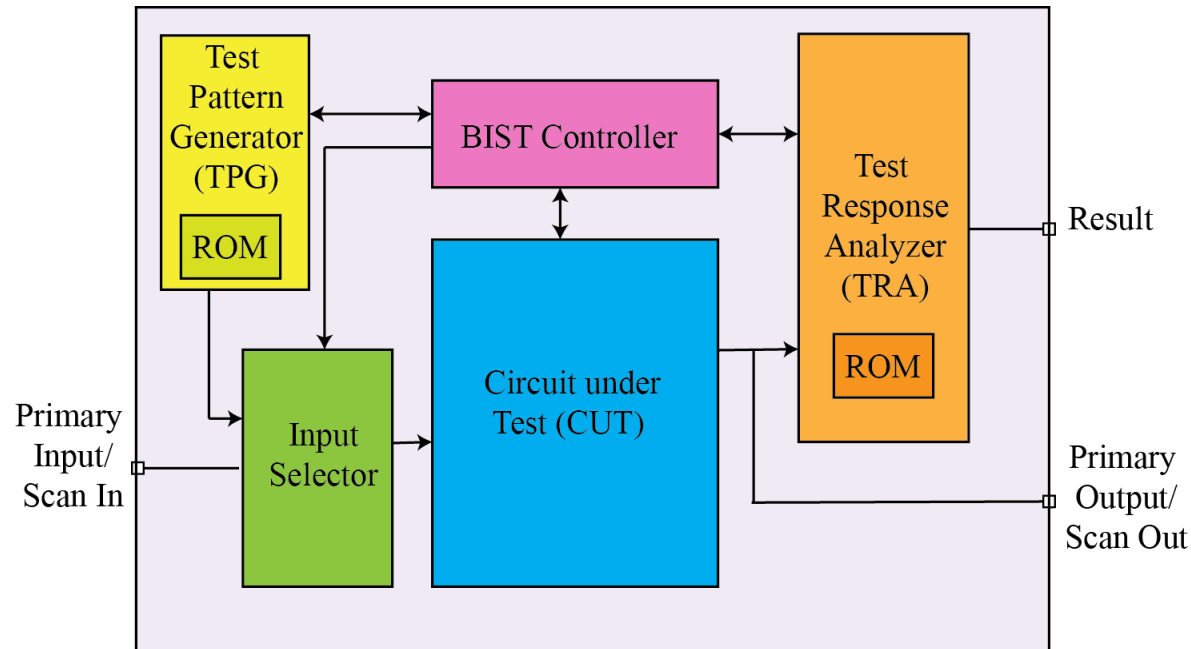
1. Contains *Pseudo-random pattern generator* inside the chip
 - Provides test vectors dynamically
 - Avoids storing voluminous test vectors
2. Performs *signature analysis* for detecting failures
 - Does not perform comparison of exact response
 - Avoids storing voluminous expected response inside the chip

Testability: Fault Coverage vs. Random Test Pattern Count



- Stuck-At Fault coverage rises to 100% logarithmically
- Reaching 90% coverage is easy using random patterns

BIST: Architecture



- **Test Pattern Generator (TPG)**

- Generates pattern for testing
- Can contain ROM

- **Test Response Analyzer (TRA)**

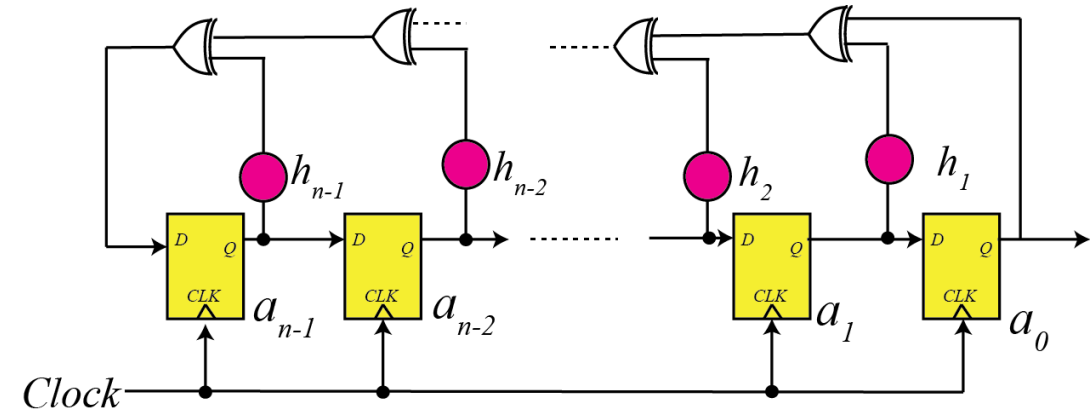
- Generates signature for the test response and compare
- ROM stores golden signature
- Compares test signature with the golden signature

- **Test Controller**

- Controls all the operations of BIST
- Strategy for overall test control is most difficult part of BIST

BIST: Test Pattern Generator

- Pseudo-Random Pattern Generator
 - Linear Feedback Shift Register (LFSR)
 - Repeatable
 - Small hardware can produce large number of “random” test patterns
- We can tap the output of any flip-flop to provide feedback.
- If we tap the output a_i of the i-th flip-flop, then h_i is taken as 1, else it is taken as 0.



$$a_0^{k+1} = a_1^k$$

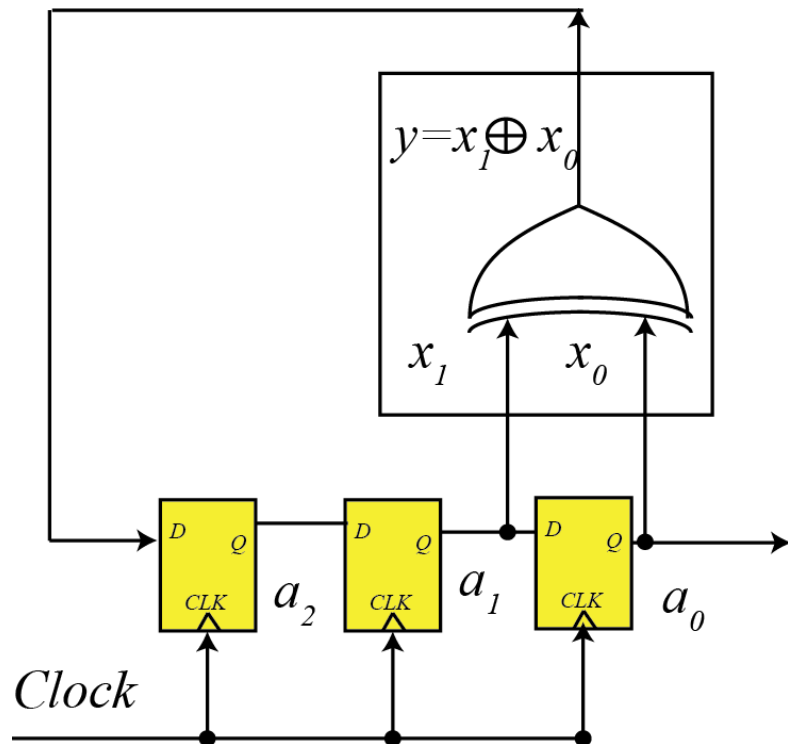
$$a_1^{k+1} = a_2^k$$

....

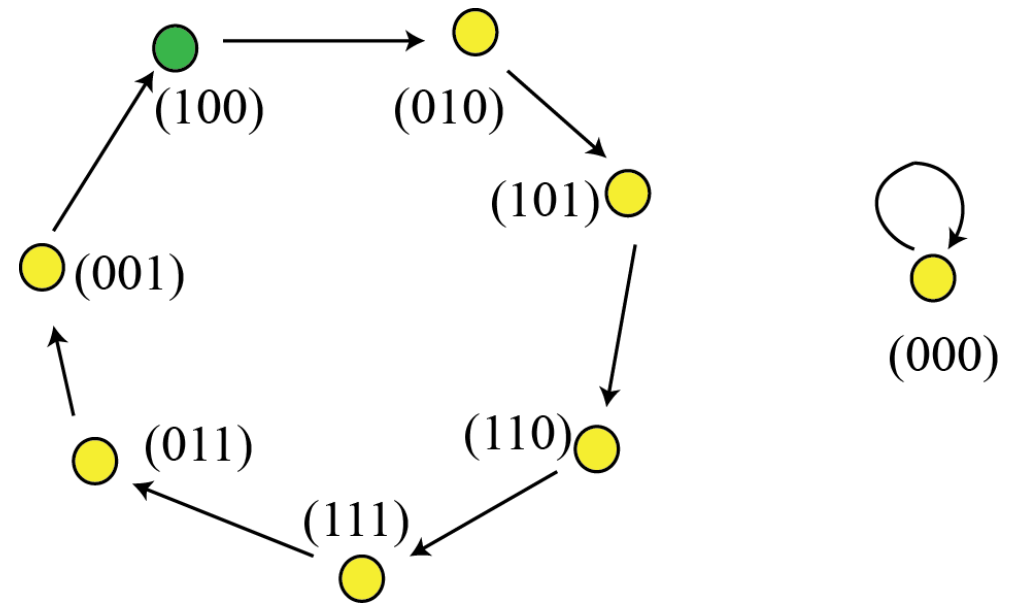
$$a_{n-2}^{k+1} = a_{n-1}^k$$

$$a_{n-1}^{k+1} = a_0^k \oplus h_1 a_1^k \oplus h_2 a_2^k \oplus \dots \oplus h_{n-1} a_{n-1}^k$$

Pattern Generator: Illustration



$$\begin{aligned}a_0^{k+1} &= a_1^k \\a_1^{k+1} &= a_2^k \\a_2^{k+1} &= a_0^k \oplus a_1^k\end{aligned}$$



Pattern Generator: Considerations

- LFSR

- Test patterns generated by an LFSR have most of the properties of random numbers
- Desirable to have long sequence so that good fault coverage is achieved

- Some faults may be not covered by random pattern generator

- Use ATPG to get deterministic test-vector and put those test-vectors in **ROM**
- Helps achieve high fault coverage with small number of test vectors

References

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.